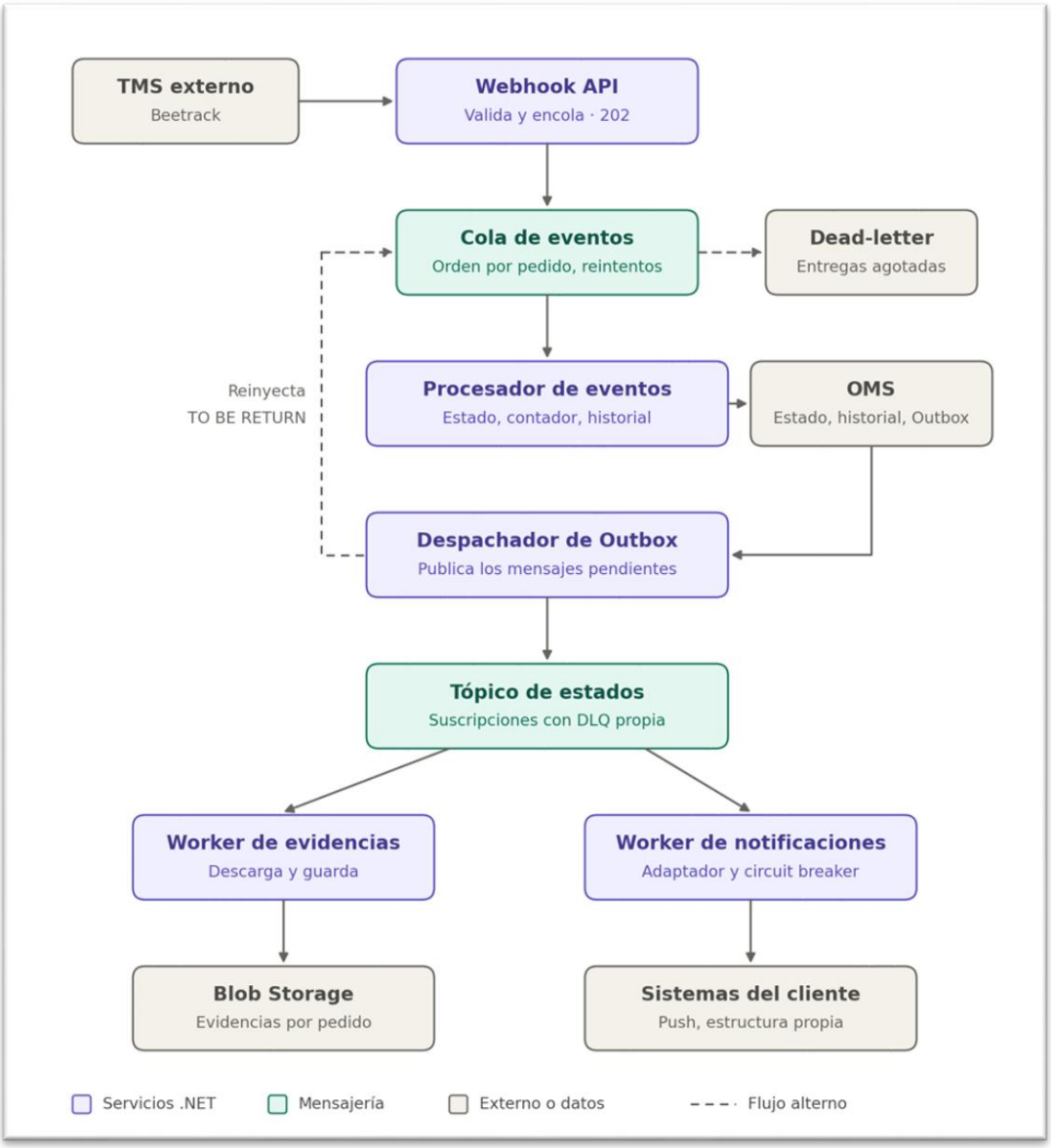
1. Diagrama de Arquitectura:



2. Descripción de componentes:

Los componentes principales se describen con su función, responsabilidades y manejo de fallos. Los componentes de apoyo se resumen en la tabla.

2.1.Webhook API

Es la puerta de entrada. Recibe el evento del TMS, lo valida, lo traduce al modelo interno, lo deja en la cola y responde. No actualiza el OMS ni descarga archivos.

- Autenticidad: exige la cabecera X-Api-Key y la compara en tiempo constante, para impedir eventos falsos sin filtrar información por el tiempo de respuesta.

- Estructura y valores: el traductor valida estado, tipo de servicio y tipo de despacho contra las listas conocidas, el número de pedido, el formato de la fecha y que cada evidencia traiga nombre y URL. Acumula todos los errores y los devuelve juntos.
- No valida reglas de negocio (existencia del pedido, estado final, duplicados): requieren consultar el OMS y corresponden al procesador.
- Respuestas: 202 con el identificador de correlación y la clave del evento; 400 si el mensaje es inválido; 401 si falla la autenticación; error 500 si no pudo encolar, para que el TMS reintente.
- Fecha: la trama no trae zona horaria, así que se asume la hora de Perú (UTC-5), configurable.
- Encola el evento ya traducido junto con el identificador de correlación (X-Correlation-Id o uno generado). En producción conviene guardar además el JSON original para auditoría.

2.2.Cola de eventos

Buffer durable entre la recepción y el procesamiento. El mensaje solo se elimina cuando el consumidor confirma que terminó.

- Absorbe picos de tráfico y caídas del OMS: los eventos se acumulan y se procesan al volver.
- Bloqueo temporal: un mensaje recibido queda bloqueado; si no se resuelve a tiempo, vuelve a estar disponible.
- Orden por pedido: mientras un mensaje de un pedido está en proceso, no se entrega otro del mismo pedido. Mensajes de pedidos distintos sí se procesan en paralelo.
- Reintentos por capas: reintento inmediato dentro del procesador, con espera creciente y solo ante fallos temporales; reentrega de la cola con espera entre intentos; y dead-letter queue al superar el máximo de entregas.
- Posposición: un consumidor puede pedir que un mensaje espere sin que cuente como intento de entrega. Lo usa el circuit breaker.

2.3.Dead-letter queue (DLQ)

Aísla los mensajes que no pudieron procesarse, sin perderlos y sin bloquear al resto. Hay una DLQ para la cola principal y otra para cada suscripción del tópico.

- Recibe mensajes que superaron el máximo de entregas y mensajes con fallos permanentes, que van directo sin gastar reintentos.
- Conserva el mensaje original, el motivo del fallo, el número de entregas, la fecha y el identificador de correlación.

- Se consulta y se reprocesa desde los endpoints de diagnóstico: el mensaje vuelve a la cola con el contador de entregas en cero.
- El reproceso es seguro porque el procesamiento es idempotente. Si se reprocesa sin corregir la causa, el mensaje vuelve a fallar y regresa a la DLQ.

2.4.Procesador de eventos

Es el cerebro de la integración y el único componente que modifica el estado del pedido en el OMS. Concentra las reglas de negocio y no ejecuta tareas lentas.

- Idempotencia: consulta la clave del evento (pedido, estado, subestado y fecha); si ya fue procesado, confirma el mensaje y termina.
- Estado final protegido: un pedido en DELIVERED o RETURNED no se modifica. El evento se registra en el historial como ignorado.
- Contador de visitas: suma 1 en DELIVERED y NOT DELIVERED. Al llegar exactamente a 3 tras un NOT DELIVERED, el dominio genera un evento TO BE RETURN.
- Commit atómico: estado, historial, marca de idempotencia y mensajes del Outbox se guardan en una sola operación: o se guarda todo o no se guarda nada.
- Concurrencia optimista: el pedido lleva un número de versión. Si otro proceso lo modificó entre la lectura y el guardado, el conflicto se trata como temporal y el reintento vuelve a leer el pedido.
- Fallos: un OMS no disponible o un conflicto de concurrencia es temporal y se reintenta. Un pedido inexistente también se trata como temporal, porque puede ser un desfase de sincronización; si persiste, termina en la DLQ.

2.5.Despachador de Outbox

Publica los mensajes que el procesador dejó pendientes en el mismo commit que el cambio de estado. Es lo que garantiza que ningún cambio guardado quede sin anunciar.

- Enruta por tipo: los cambios de estado van al tópico y los eventos TO BE RETURN vuelven a la cola principal.
- Publica en orden y se detiene ante el primer fallo, para que el cambio de estado salga siempre antes que el TO BE RETURN que originó.
- Marca cada mensaje como despachado después de publicarlo. Si publica, pero cae antes de marcarlo, se reenvía: la entrega es al menos una vez, y los consumidores son idempotentes.

2.6.Tópico de estados

Canal de publicación/suscripción donde se anuncia cada cambio de estado ya guardado. Cada suscripción actúa como una cola propia, con su bloqueo, sus reintentos y su DLQ.

- Fan-out: se publica una vez y cada suscriptor recibe su copia, sin que el procesador conozca a los consumidores.
- Filtros: evidencias solo recibe los seis hitos; notificaciones recibe todos los cambios, incluido el TO BE RETURN.
- El mensaje incluye pedido, cliente, estado, subestado, fecha, datos para notificar, referencias a evidencias, contador de visitas y la clave del evento.
- Un consumidor caído no afecta a los demás: su suscripción acumula y reintenta por separado.

2.7.Worker de evidencias

Descarga las evidencias digitales (fotos, firmas) alojadas en el TMS y las guarda en Blob Storage, asociadas al pedido y al evento.

- Actúa solo en COLLECTED, NOT COLLECTED, DELIVERED, NOT DELIVERED, RETURNED y NOT RETURNED.
- Ruta de almacenamiento: pedido / fecha del evento en UTC y estado / nombre del archivo. El nombre se sanea, porque viene de un tercero.
- Controles antes de guardar: extensiones permitidas (.jpg, .jpeg, .png, .pdf), tamaño máximo de 5 MB y rechazo de archivos vacíos.
- Idempotencia por archivo y por mensaje: si el archivo ya está guardado no se descarga de nuevo, de modo que un fallo parcial se completa sin duplicar.
- Procesa todas las evidencias, aunque una falle. Un fallo temporal devuelve el mensaje y el reintento solo completa lo que falta; un fallo permanente (archivo inexistente) va a la DLQ de evidencias sin impedir que las demás se guarden.
- Un catálogo registra la asociación entre pedido, evento, archivo y ruta.

2.8.Worker de notificaciones

Avisa al cliente cada cambio de estado, traduciendo el mensaje interno a la estructura que ese cliente espera.

- Configuración por cliente: formato, destino y estados que quiere recibir. Se guarda fuera del código y de los mensajes.
- Adaptador por formato: resuelto mediante un registro. Hay dos de ejemplo: uno estándar (claves en inglés, fecha ISO 8601) y uno antiguo (claves en español, fecha dd/MM/yyyy). Agregar un cliente no modifica el worker.

- Idempotencia: marca lo ya enviado y envía un identificador único en cada notificación, para que el cliente pueda descartar duplicados por su cuenta.
- Circuit breaker por cliente: tres fallos temporales seguidos abren el circuito. Mientras está abierto no se llama al cliente y el mensaje se pospone sin consumir entregas; al terminar la espera se permite una llamada de prueba.
- Fallos permanentes (cliente sin configuración, formato desconocido, rechazo del cliente) van a la DLQ de notificaciones y no abren el circuito, porque indican que el cliente responde, no que esté caído.

3. Justificación de patrones de integración

Aceptar rápido, procesar después

Problema. El TMS espera una respuesta ágil. Si el procesamiento completo ocurre dentro de la petición, un OMS lento provoca timeouts y el TMS reenvía el evento.

Patrón y decisión. El Webhook API valida lo mínimo, encola y responde 202. Se descarta procesar de forma síncrona y responder 200, porque acopla la disponibilidad del TMS a la de todo el sistema.

Costo asumido. El TMS no sabe si el procesamiento terminó. Se compensa con historial, DLQ, diagnóstico y alertas propias.

Cola: nivelación de carga y consumidores competidores

Publish-Subscribe (fan-out)

Problema. Un mismo cambio de estado dispara varias acciones que no deben depender entre sí.

Patrón y decisión. El despachador publica un hecho en un tópico y cada suscripción recibe su copia, con filtro, reintentos y DLQ propios. Se usa cola para repartir trabajo y tópico para difundir hechos. Se descarta que el procesador llame a cada worker, porque lo acopla a todos y su velocidad la marcaría el más lento.

Costo asumido. Más piezas que operar, y cada consumidor debe ser idempotente.

Reintentos por capas y Dead Letter Channel

Problema. Los fallos temporales son inevitables, pero reintentar sin límite bloquea el flujo y no reintentar pierde datos.

Patrón y decisión. Tres capas: reintento inmediato con espera creciente (solo ante fallos temporales), reentrega de la cola con espera entre intentos y, al

agotarse las entregas, aislamiento en la DLQ. Los fallos permanentes van directo a la DLQ sin gastar reintentos, y los mensajes venenosos no bloquean al resto.

Costo asumido. Hay que operar la DLQ (alertas, revisión, reproceso). Una DLQ ignorada es un fallo escondido.

Circuit Breaker con posposición

Problema. Un cliente con su sistema caído genera reintentos constantes que consumen recursos y retrasan a los demás. Peor aún: si esos reintentos consumieran entregas, los avisos acabarían en la DLQ aunque el mensaje no tenga ningún defecto.

Patrón y decisión. Un circuit breaker por cliente: tras tres fallos temporales seguidos se deja de llamar al cliente durante un tiempo y luego se prueba con una sola llamada. Mientras está abierto, el mensaje se pospone sin contar como intento de entrega. El reintento envuelve al breaker, de modo que cada intento cuenta para abrirlo y, con el circuito abierto, el reintento no insiste. Los rechazos permanentes no cuentan: el cliente responde, no está caído.

Costo asumido. Más configuración y un caso límite: un cliente caído de forma indefinida mantiene sus avisos esperando. En producción se resolvería con un tiempo de vida del mensaje que lo envíe a la DLQ al caducar.

Idempotent Receiver

Problema. Con entrega al menos una vez, un mensaje puede llegar dos veces. Aquí sería grave: duplicaría una visita, podría disparar un TO BE RETURN indebido o avisaría dos veces al cliente.

Patrón y decisión. Cada consumidor se protege a su manera. El procesador usa la clave del evento; el worker de evidencias, la ruta del archivo; el de notificaciones, la clave del evento más un identificador único que el cliente puede usar para descartar duplicados. Es el patrón más importante del diseño: hace seguros los reintentos, la DLQ y el reproceso.

Costo asumido. Hay que guardar las claves y definir con cuidado qué identifica a un evento.

Transactional Outbox

Problema. El procesador guarda el estado y luego publica el mensaje. Si falla entre ambos pasos, el pedido cambia pero nadie se entera: no se descargan evidencias ni se notifica.

Patrón y decisión. El mensaje a publicar se guarda en el mismo commit atómico que el cambio de estado, y un despachador aparte lo envía. Se descarta publicar directamente tras guardar, que es lo más común y lo más frágil.

Costo asumido. Un proceso adicional, un pequeño retraso en la publicación y entrega al menos una vez. A cambio, nunca hay estado sin su evento.

Adapter / Strategy y Message Translator

Problema. Cada cliente tiene su propia estructura de notificación y habrá más con el tiempo.

Patrón y decisión. Un adaptador por cliente traduce el mensaje interno a su formato, y un registro lo resuelve según la configuración del cliente. Agregar un cliente no modifica el worker (principio abierto/cerrado). Se descartan los condicionales por cliente.

Costo asumido. Más clases, compensadas por claridad y facilidad de prueba.

Anti-Corruption Layer y modelo canónico

Problema. El formato del TMS no es el interno y puede cambiar sin aviso.

Patrón y decisión. En la entrada, el traductor convierte la trama del TMS en un modelo interno y el resto del sistema solo conoce ese modelo. Un cambio de proveedor o de formato queda contenido en un solo punto.

Costo asumido. Una traducción adicional, que vale la pena cuando el emisor es externo y no se controla.

Orden y concurrencia por pedido

Problema. Dos eventos del mismo pedido procesados a la vez pueden dejar mal el contador, y los eventos desordenados pueden producir estados incoherentes.

Patrón y decisión. Dos defensas combinadas: la cola no entrega un mensaje mientras haya otro del mismo pedido en proceso, y el pedido lleva un número de versión que permite detectar modificaciones simultáneas al guardar.

Costo asumido. La entrega ordenada limita el paralelismo dentro de un pedido, pero se mantiene entre pedidos distintos. Reprocesar desde la DLQ coloca el mensaje al final de la cola, de modo que el orden puede alterarse; la regla de estado final protege lo más delicado.

Máquina de estados en el dominio

Problema. Las reglas de estado final, contador de visitas y auto-emisión de TO BE RETURN son lo más importante del negocio y deben ser correctas.

Patrón y decisión. Esas reglas viven en una capa de dominio aislada, sin dependencias de colas, bases de datos ni reloj. El resultado de aplicar un evento es un valor (resultado, historial y evento de seguimiento), así que se prueban con facilidad.

Costo asumido. Una capa más de organización, justificada por el valor de esas reglas.

Correlation Identifier

Problema. Un evento atraviesa la cola, el procesador, el Outbox, el tópico y dos workers. Sin un identificador común es muy difícil reconstruir qué pasó con él.

Patrón y decisión. El webhook toma el identificador de la cabecera X-Correlation-Id o genera uno, y viaja con el mensaje por cada etapa. Se devuelve en la respuesta del webhook y se conserva en la DLQ.

Costo asumido. Prácticamente ninguno.

Ports & Adapters en el proyecto .NET

Problema. El caso permite no conectar servicios reales, pero la arquitectura debe ser creíble.

Patrón y decisión. Cada dependencia externa (OMS, cola, tópico, almacenamiento, notificador) se define como una interfaz con una implementación en memoria que reproduce su comportamiento real. Sustituirla por un servicio de mensajería o una base de datos reales no tocaría la lógica de negocio.

Costo asumido. Hay que simular el comportamiento de cada dependencia (bloqueo, reentrega, DLQ), pero eso hace visible la lógica de reintentos al evaluar el proyecto.